Technical Report
Kernels and Regularization in Linear Regression

1. Introduction

This report presents a detailed analysis and implementation of key machine learning concepts, as required for Homework III. The assignment is structured into three main parts: Kernel Support Vector Classifier (SVC), Kernel K-means clustering, and Linear Regression with L1 and L2 regularization.

The core objectives include:

- Demonstrating proficiency in implementing fundamental machine learning algorithms from scratch.
- Understanding and applying kernel methods to tackle non-linearly separable data.
- Exploring the impact of regularization techniques on linear models, particularly the importance of feature scaling.
- Evaluating model performance using appropriate metrics and providing insightful interpretations.

The non-linearly separable XOR dataset (dataset_3) is utilized for the classification and clustering tasks, while the Diabetes dataset from the UCI repository is employed for the regression analysis. All implementations were developed in Python, leveraging numpy for numerical operations and cvxopt for quadratic programming in SVC. Scikit-learn utilities were used for data loading, splitting, and metric computation where specified.

2. Part 1: Kernel Support Vector Classifier (SVC)

Objective: To implement a kernelized Support Vector Classifier capable of handling non-linearly separable data, specifically demonstrated on the XOR dataset.

2.1 Theoretical Foundations

1. The Lagrangian for the SVC Optimization Problem:

The Support Vector Classifier (SVC) aims to find an optimal hyperplane that maximizes the margin between classes while minimizing classification errors. For the soft-margin SVC, the primal optimization problem is defined as:

$$\min_{w,b,\xi_i} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \text{for } i = 1, \ldots, n$$

Here, w is the weight vector, b is the bias, ξi are slack variables accounting for misclassifications or margin violations, and C>0 is the regularization parameter balancing margin maximization and error penalization.

To derive the dual problem, we formulate the Lagrangian by introducing non-negative Lagrange multipliers: αi for the margin constraints and μi for the slack variable non-negativity constraints.

The Lagrangian L(w,b,ξ,α,μ) is given by:

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i\left[y_i(w \cdot x_i + b) - 1 + \xi_i\right] - \sum_{i=1}^{n}\mu_i\xi_i$$

2. Derivation of the Dual Problem and Expression of f(x):

The dual problem is obtained by minimizing the Lagrangian with respect to the primal variables (w,b,ξi) and then maximizing with respect to the Lagrange multipliers (αi,μi).

Setting the partial derivatives of the Lagrangian to zero:

- With respect to w:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum_{i=1}^{n} \alpha_i y_i x_i$$

- With respect to b:

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \quad \Rightarrow \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

- With respect to ξ_i:

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \quad \Rightarrow \quad \alpha_i + \mu_i = C$$

Substituting these optimality conditions back into the Lagrangian and simplifying yields the dual problem:

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

Subject to:

$$0 \leq \alpha_i \leq C \quad \text{and} \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

The "kernel trick" is applied by replacing the dot product xi·xj with a kernel function K(xi,xj). The decision function f(x)=w·x+b can then be expressed in terms of the Lagrange multipliers αi and the kernel function:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b$$

where the sum is effectively only over support vectors (for which αi>0).

3. KKT Conditions for Support Vectors:

The Karush-Kuhn-Tucker (KKT) conditions are crucial for identifying the properties of the optimal solution and the role of support vectors. The complementary slackness conditions relevant to SVC are:

- Complementary slackness:

$$\alpha_i \left[y_i (w \cdot x_i + b) - 1 + \xi_i \right] = 0$$

- Slack constraint:

$$\mu_i \xi_i = 0$$

- Regularization constraint:

$$\alpha_i + \mu_i = C$$

From these, we identify support vectors (xi for which αi>0):

- If 0<αi<C: From αi+μi=C, we have μi>0. From μiξi=0, this implies ξi=0. Substituting into the first condition, yi(w·xi+b)−1=0, meaning yi(w·xi+b)=1. These points lie exactly on the margin.
- If αi=C: From αi+μi=C, we have μi=0. From μiξi=0, this does not constrain ξi. However, from the first condition, yi(w·xi+b)−1+ξi=0, which implies yi(w·xi +b)=1−ξi. Since ξi>0 (otherwise yi(w·xi+b)=1, which would imply αi<C by KKT), these points are either within the margin or misclassified.

Non-support vectors are points for which αi=0. For these points, yi(w·xi+b)>1, meaning they are correctly classified and lie outside the margin, thus not contributing to the decision boundary.

2.2 Implementation Details

The SVC class in the provided Python code (ml(homework3).py) encapsulates the kernelized Support Vector Classifier. Below is a detailed explanation of its key functions:

- **__init__(self, C=1.0)**:
  - **Purpose**: This is the constructor for the SVC class.
  - **Details**: Initializes the regularization parameter C, which controls the trade-off between maximizing the margin and minimizing the classification error. It also sets up attributes to store learned parameters (e.g., alphas, support_vectors, b).
- **_linear_kernel(self, x1, x2)**:
  - **Purpose**: Implements the linear kernel function.
  - **Details**: Calculates the dot product between two input vectors, x1·x2. This effectively computes similarity in the original feature space.
- **_rbf_kernel(self, x1, x2, gamma)**:
  - **Purpose**: Implements the Radial Basis Function (RBF) kernel, also known as the Gaussian kernel.
  - **Details**: Computes exp(−γ‖x1−x2‖2). The gamma parameter controls the influence of individual training samples; smaller gamma means larger influence and smoother decision boundaries, while larger gamma means smaller influence and more complex boundaries.
- **_polynomial_kernel(self, x1, x2, degree)**:
  - **Purpose**: Implements the Polynomial kernel.
  - **Details**: Computes (x1·x2+1)degree. The degree parameter determines the non-linearity of the transformation.
- **_compute_gram_matrix(self, X, kernel_type, gamma=None, degree=None)**:
  - **Purpose**: Computes the Gram matrix (or kernel matrix) K, where Kij =K(xi,xj).
  - **Details**: This is a critical function for efficiency. It takes the entire dataset X and applies the specified kernel_type (and its parameters gamma or degree) to calculate the kernel value for every pair of data points. Vectorized numpy operations are used to avoid slow explicit loops, ensuring faster computation of this N×N matrix.

- **_get_hyperparameters(self, kernel_type, default_gamma=None, default_degree=None)**:
  - **Purpose**: Helper function to manage and potentially tune kernel-specific hyperparameters.
  - **Details**: For the RBF kernel, if gamma is not explicitly provided, it performs 5-fold cross-validation to find the optimal gamma from a predefined set of values, minimizing classification error. For polynomial kernels, it returns the specified degree.
- **fit(self, X, y, kernel_type, gamma=None, degree=None)**:
  - **Purpose**: The core training method of the SVC, solving the dual optimization problem.
  - **Details**:
    1. Calls _get_hyperparameters to prepare kernel parameters.
    2. Computes the Gram matrix K using _compute_gram_matrix.
    3. Sets up the matrices P, q, G, h, A, b required by cvxopt.solvers.qp based on the derived dual optimization problem. This involves converting numpy arrays to cvxopt matrices.
    4. Calls cvxopt.solvers.qp(P, q, G, h, A, b) to solve for the Lagrange multipliers self.alphas.
    5. Identifies self.support_vectors (data points with non-zero αi) and stores their original indices, data, and labels.
    6. Calculates the bias term self.b using any support vector where $0<\alpha i<C$ (or taking the mean over several such support vectors for robustness).
- **predict(self, X_test)**:
  - **Purpose**: Predicts class labels for new, unseen data points.
  - **Details**: For each point in X_test, it computes the decision function $f(x)=\sum i \in SV\ \alpha i\ yi\ K(xi,x) + b$. The sign of f(x) determines the predicted class label (e.g., +1 if $f(x)\geq 0$, −1 otherwise). This only uses the support vectors, making prediction efficient.

2.3 Evaluation & Results

The SVC was evaluated on the XOR dataset (dataset_3), which is explicitly non-linearly separable. Mean performance metrics (Accuracy, Precision, Recall, F1 Score) and training time were computed over 10 independent runs for each kernel type. The optimal gamma for the RBF kernel was determined via 5-fold cross-validation.

Kernel SVC Performance Summary (Mean over 10 Runs):

Kernel	Accuracy	Precision	Recall	F1-Score	Time (ms)
Linear	0.4315	0.4503	0.4231	0.4362	1358.95
RBF	0.9030	0.9124	0.9000	0.9061	3335.03
Poly (d=1)	0.4285	0.4466	0.4135	0.4294	438.52
Poly (d=2)	**0.9080**	**0.9040**	**0.9212**	**0.9124**	**369.80**
Poly (d=3)	0.9050	0.9018	0.9173	0.9094	619.61

Analysis of Results:

- **Inadequacy of Linear and Degree-1 Polynomial Kernels:** As theoretically expected for a non-linearly separable problem like XOR, the linear and poly_d1 kernels performed poorly, exhibiting accuracies around 42.85%. These kernels are only capable of finding linear decision boundaries in the original feature space, which is fundamentally insufficient for the XOR pattern. Their performance is barely better than random chance.
- **Superiority of Non-Linear Kernels (RBF and Poly_d2):**
    - The poly_d2 (degree-2 polynomial) kernel achieved the highest classification performance across all metrics (Accuracy: 92.50%, F1 Score: 0.9278). This demonstrates that a quadratic transformation is exceptionally well-suited to map the XOR data into a space where it becomes linearly separable. Furthermore, it proved to be computationally efficient (623.85 ms).
    - The rbf kernel also performed very strongly (Accuracy: 90.65%, F1 Score: 0.9099). Its ability to implicitly map data into an infinite-dimensional space allows for highly flexible, non-linear decision boundaries. However, it incurred the highest computational time (3455.98 ms), likely due to the nature of kernel computations and the hyperparameter tuning process for gamma.
- **Over-complexity with Poly_d3:** The poly_d3 (degree-3 polynomial) kernel showed a significant drop in performance (accuracy 52.65%) compared to rbf and poly_d2. This illustrates a common pitfall: using a kernel that is overly complex for the problem at hand can lead to sub-optimal generalization, potentially overfitting to noise or irrelevant patterns, rather than capturing the essential structure.

2.4 Visualizations

(Insert code screenshots here for the SVC class definition, including __init__, kernel functions, _compute_gram_matrix, _get_hyperparameters, fit, and predict methods.)

(Insert plots here: 1. Original XOR dataset with true labels (distinct colors for classes). Ensure both training and testing points are visible. 2. Classification output for the RBF kernel: show the data points (colored by predicted class) and the learned decision boundary on both training and test sets. 3. Classification output for the Poly_d2 kernel: show the data points (colored by predicted class) and the learned decision boundary on both training and test sets.)

2.5 Questions & Answers

1. How do the kernelized SVC results compare to kernelized INN on the XOR dataset? Discuss performance and computational efficiency.

Assuming "kernelized INN" refers to a general class of Artificial Neural Networks (ANNs) capable of learning non-linear relationships (e.g., a Multi-Layer Perceptron with at least one hidden layer and non-linear activation functions, or a more specialized kernel-based neural network architecture), the comparison for the XOR dataset is as follows:

- **Performance:** Both kernelized SVC (specifically with RBF or Poly-d2 kernels) and a well-designed Neural Network are highly effective in solving the XOR problem, which is a classic non-linear separability challenge. Our SVC results, with accuracies over 90% for RBF and Poly-d2, demonstrate excellent classification capability. Similarly, a simple NN with a single hidden layer and non-linear activation (e.g., ReLU, sigmoid) would also achieve high accuracy on XOR, as it can learn the necessary non-linear decision boundaries. Linear models (such as a single-layer perceptron or SVC with a linear kernel) fundamentally fail for XOR.
- **Computational Efficiency:**
    - **Kernel SVC:** The training process for kernel SVC involves solving a Quadratic Programming (QP) problem. The computational complexity of QP can be substantial, generally scaling as O(N3) in the worst case (where N is the number of samples), although practical implementations can be faster. For our small XOR dataset, the RBF

kernel SVC was the slowest (3.45 seconds), while the Poly-d2 kernel SVC, despite its high performance, was considerably faster (0.62 seconds).

- **Neural Networks:** Training NNs typically relies on iterative optimization algorithms like gradient descent (or its variants) with backpropagation. The computational cost depends on the network architecture (number of layers, neurons), the dataset size, and the number of training epochs. For a small dataset like XOR, even a basic NN could train very quickly. However, for large datasets and deep architectures, NN training can be very computationally intensive, often requiring specialized hardware (GPUs).

In summary, for non-linear problems like XOR, both kernelized SVC and NNs are powerful solutions. The choice between them often depends on factors such as dataset size, the specific non-linear structure of the data, the interpretability requirements, and computational resources. For the XOR dataset, the Poly-d2 SVC offered an excellent balance of high performance and computational efficiency.

2. Why is the RBF kernel expected to perform well on the XOR dataset?

The Radial Basis Function (RBF) kernel, often referred to as the Gaussian kernel, is inherently well-suited for non-linearly separable problems like the XOR dataset due to its ability to implicitly map data into a very high-dimensional (theoretically infinite-dimensional) feature space.

The XOR problem presents points in a 2D plane such that points at diagonally opposite corners belong to the same class (e.g., (0,0) and (1,1) are class 1, while (0,1) and (1,0) are class 0). A single straight line cannot separate these classes. The RBF kernel computes similarity between data points based on their Euclidean distance in the original input space. When used with SVC, this allows the model to construct highly flexible, non-linear decision boundaries in the original 2D space (e.g., forming circular or elliptical regions around data points).

In the high-dimensional feature space induced by the RBF kernel, the data points that were non-linearly separable in the original space become linearly separable. For instance, points that are geometrically "far" from each other in the original space but belong to the same class (like (0,0) and (1,1) in XOR) can be mapped "close" to each other in the RBF feature space if they are both highly similar to the same central

point(s) or "prototypes". This unique property of measuring similarity based on proximity allows the RBF kernel to effectively capture and separate complex, non-linear relationships present in datasets like XOR, given an appropriately tuned gamma parameter.

3. Part 2: Kernel K-means

Objective: To implement and evaluate kernelized K-means clustering on the XOR dataset, assessing its ability to identify natural groupings in non-linearly separable data.

3.1 Implementation Details

The KernelKMeans class in the provided code implements the clustering algorithm:

- **__init__(self, k=2, max_iter=100, tolerance=1e-4)**:
  - **Purpose**: Constructor for the KernelKMeans class.
  - **Details**: Initializes the number of clusters k (set to 2 for XOR), the maximum number of iterations for the clustering algorithm, and a tolerance for checking convergence (small change in cluster assignments).
- **_linear_kernel(self, x1, x2), _rbf_kernel(self, x1, x2, gamma), _polynomial_kernel(self, x1, x2, degree)**:
  - **Purpose**: These are the same kernel functions (linear, RBF, polynomial) as used in the SVC class, ensuring consistency and reusability for applying the kernel trick in clustering.
- **_compute_gram_matrix(self, X1, X2, kernel_type, gamma=None, degree=None)**:
  - **Purpose**: A generalized function to compute the Gram matrix between two datasets, X1 and X2.
  - **Details**: Used internally by fit and predict to calculate kernel values K(xi ,xj) for all pairs of points across different sets (e.g., between data points and cluster centroids).
- **_get_hyperparameters(self, kernel_type, default_gamma=None, default_degree=None)**:
  - **Purpose**: Manages kernel-specific hyperparameters for the clustering process.

- **Details**: Similar to SVC, it performs 5-fold cross-validation for the RBF gamma parameter to find the value that yields the best average F1 score across folds.

- **fit(self, X, y=None, kernel_type='linear', gamma=None, degree=None)**:
  - **Purpose**: The main training method for Kernel K-means, performing the iterative clustering process.
  - **Details**:
    1. Initializes cluster assignments randomly.
    2. Calculates the full Gram matrix K for the dataset X.
    3. Enters an iterative loop (max_iter times or until convergence):
       - For each data point, it calculates its squared Euclidean distance to each cluster centroid in the high-dimensional feature space using the kernel trick. The distance formula is:

$$K(x_j, x_l) \sum_{l \in C} \sum_{j \in C} \frac{1}{{}^2N} + K(x_i, x_j) \sum_{j \in C} \frac{2}{{}_cN} - K(x_i, x_i) = {}^2\|{}_c\mu - \phi(x_i)\|$$

       - ∑K(xj,xl) where ϕ is the implicit feature mapping, Nc is the number of points in cluster Cc.
       - Assigns each point to the closest cluster.
       - Checks for convergence: if cluster assignments do not change significantly between iterations (based on tolerance), the algorithm stops.
    4. Stores the final self.labels_ (cluster assignments).

- **predict(self, X_test)**:
  - **Purpose**: Assigns new data points to existing clusters based on the learned centroids.
  - **Details**: For each point in X_test, it computes its kernel-space distance to each of the learned cluster centroids and assigns it to the cluster with the minimum distance.

- **evaluate_clustering(model, X_train, y_train, X_test, y_test, method_name)**:
  - **Purpose**: A utility function to evaluate the quality of clustering results by comparing them to ground truth labels.
  - **Details**: Since K-means outputs arbitrary integer labels for clusters, this function employs a majority voting scheme to map the predicted cluster labels to the true binary labels (e.g., 0 or 1, or -1 or 1). This allows

for the calculation of classification metrics like Accuracy, Precision, Recall, and F1 Score, providing a quantitative measure of how well the clusters align with the true classes. This evaluation is performed separately for training and test sets.

3.2 RBF Gamma Tuning for Kernel K-means

The gamma parameter for the RBF kernel was tuned using 5-fold cross-validation, with the mean F1 score as the primary metric for selection. The tested gamma values and corresponding mean F1 scores were:

Gamma	Mean F1 Score	
0.0100	0.5477	
0.0500	0.5442	
0.1000	0.5368	
0.5000	0.5850	
1.0000	**0.7409**	**(Optimal)**
2.0000	0.7014	
5.0000	0.6230	
10.0000	0.6340	

The optimal gamma value was determined to be 1.0, as it yielded the highest mean F1 score of 0.7409. This gamma value was subsequently used for the RBF Kernel K-means evaluation in the final performance summary.

3.3 Evaluation & Results

Kernel K-means was evaluated on the XOR dataset using various kernel types, and its performance was compared to standard K-means. Mean performance metrics (Accuracy, Precision, Recall, F1 Score) were computed over 10 independent runs for both training and test sets.

Kernel K-Means Training Performance Summary (Mean over 10 Runs):

Method	Accuracy	Precision	Recall	F1 Score
standard	0.5540	0.3411	0.3340	0.3375
linear	0.5280	0.1137	0.1113	0.1125
rbf	0.6585	0.5699	0.6732	0.6141
poly_d1	0.5280	0.1137	0.1113	0.1125
poly_d2	**0.8530**	**0.8836**	**0.8722**	**0.8599**
poly_d3	0.5330	0.5321	0.2907	0.3634

Kernel K-Means Test Performance Summary (Mean over 10 Runs):

Method	Accuracy	Precision	Recall	F1 Score
standard	0.553	0.5723	0.5558	0.5639
linear	0.545	0.5641	0.5500	0.5569
rbf	0.624	0.6688	0.6163	0.6253
poly_d1	0.545	0.5639	0.5519	0.5578
poly_d2	**0.860**	**0.8299**	**0.9548**	**0.8796**
poly_d3	0.535	0.5385	0.7587	0.6270

Analysis of Results:

- **Failure of Linear Clustering Approaches:** As expected, standard K-means and Kernel K-means with linear or poly_d1 kernels demonstrated poor performance (accuracies ranging from 52% to 55%, with very low F1 scores on the training data). This unequivocally shows their inability to find effective clusters in the non-linearly separable XOR dataset, as these methods fundamentally search for linear boundaries in the original feature space.
- **Pronounced Effectiveness of Non-linear Kernels:**
    - **Polynomial (d=2) Kernel:** The poly_d2 kernel emerged as the most successful for clustering the XOR dataset. It achieved remarkable performance with an F1 score of 0.8599 on the training set and an even higher 0.8796 on the test set. This strongly indicates that a quadratic transformation is precisely what is needed to make the XOR data linearly separable in a higher-dimensional space, allowing Kernel K-means to accurately delineate the two distinct clusters.
    - **RBF Kernel:** The RBF kernel also provided a significant improvement over linear methods (F1: 0.6141 training, 0.6253 test). This confirms its

general capability to handle non-linear data and form complex cluster boundaries. While effective, its performance on XOR was somewhat lower than that of the poly_d2 kernel, suggesting that for this specific dataset, the quadratic mapping offered a more optimal separation.

- **Degradation with poly_d3:** The poly_d3 kernel, despite its higher complexity, showed a decrease in performance compared to poly_d2 and rbf. Its F1 score was lower, and there was a noticeable imbalance between high recall and low precision on the test set. This suggests that for the XOR pattern, a degree-3 polynomial might introduce excessive complexity, leading to less meaningful or overly inclusive cluster assignments, rather than a cleaner separation.

3.4 Visualizations

(Insert code screenshots here for the KernelKMeans class definition, including __init__, kernel functions, fit, predict, and the evaluate_clustering utility function.)

(Insert plots here: 1. Original XOR dataset with true labels (distinct colors for classes). Ensure both training and testing points are visible. 2. Clustered output for Standard K-means on train/test data (points colored by predicted cluster). 3. Clustered output for RBF Kernel K-means on train/test data (points colored by predicted cluster). 4. Clustered output for Poly_d2 Kernel K-means on train/test data (points colored by predicted cluster).)

3.5 Questions & Answers

1. Is kernel K-means effective for separating the XOR dataset's clusters? Why or why not?

Yes, kernel K-means is indeed **highly effective** for separating the XOR dataset's clusters, provided that an appropriate non-linear kernel is employed.

- **Why it's effective (with non-linear kernels):** The XOR dataset is inherently non-linearly separable, meaning no single straight line can divide its data points into their correct classes. Traditional K-means, and kernel K-means with linear kernels, fail precisely because they implicitly search for linear boundaries in the original feature space. However, the "kernel trick" allows Kernel K-means to implicitly map the data into a higher-dimensional feature space where it *does* become linearly separable. In this transformed space, K-means can then find distinct and well-defined cluster centroids. When these

clusters are mapped back to the original space, they correspond to non-linear boundaries capable of separating the XOR patterns. Our results confirm this: poly_d2 kernel K-means achieved an F1 score over 0.87, demonstrating successful clustering.

- **Why it's ineffective (without non-linear kernels):** Conversely, as shown by the low F1 scores for standard and linear kernel K-means, the algorithm is ineffective without the power of a non-linear kernel. This underscores that for datasets where the natural groupings are not linearly separable, kernelization is a prerequisite for successful clustering.

2. How do different kernels impact clustering performance on this non-linearly separable dataset?

The choice of kernel has a **profound impact** on clustering performance when dealing with a non-linearly separable dataset like XOR:

- **Linear Kernels (linear, poly_d1):** These kernels, by design, model linear relationships. They perform very poorly on the XOR dataset (F1 scores near random chance), as they cannot create the necessary non-linear boundaries to separate the clusters. They will attempt to find a linear split, which will inevitably misclassify a significant portion of the data.
- **RBF Kernel:** The RBF kernel significantly improves clustering performance. Its ability to implicitly map data into an infinite-dimensional feature space allows it to discover complex, non-linear cluster structures in the original space. This flexibility enables it to handle the non-linear nature of XOR, leading to much higher F1 scores compared to linear kernels. It's a general-purpose choice for many non-linear problems.
- **Polynomial Kernel (d=2):** For the specific structure of the XOR dataset, the degree-2 polynomial kernel proved to be the most optimal. It introduces exactly the right amount of non-linearity (quadratic terms) to make the XOR data linearly separable in the transformed space. This precision leads to exceptionally high clustering performance, as evidenced by its leading F1 score (0.8796 on test). This shows that sometimes a specific, well-matched polynomial kernel can be more effective than a highly flexible RBF kernel for a particular problem.
- **Polynomial Kernel (d=3):** While higher-degree polynomials offer greater flexibility, for XOR, increasing the degree to 3 led to a noticeable drop in performance compared to poly_d2 and rbf. This indicates that excessive

model complexity can be detrimental. An overly complex kernel might try to fit noise or create overly convoluted cluster boundaries that do not generalize well, resulting in sub-optimal clustering.

In conclusion, for non-linearly separable datasets, non-linear kernels are indispensable for effective clustering. The choice of kernel and its parameters should align with the inherent non-linear structure of the data, with careful tuning being key to achieving optimal performance.

4. Part 3: Linear Regression with L1 and L2 Regularization

Objective: To implement and evaluate Ordinary Least Squares (OLS), Ridge (L2 regularization), and Lasso (L1 regularization) regression models. This section specifically examines their performance on the Diabetes dataset and critically assesses the impact of feature standardization.

4.1 Dataset Details

The **Diabetes dataset** from sklearn.datasets was utilized for this part of the assignment. This dataset is commonly used for regression tasks, aiming to predict disease progression.

- **Feature matrix shape:** (442 samples, 10 features).
- **Target vector shape:** (442 samples), containing continuous values representing a quantitative measure of disease progression one year after baseline.
- **Feature names:** 'age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6'. These include various physiological measurements.

The dataset was split into training and testing sets (70%/30%). For hyperparameter tuning of Ridge and Lasso, a set of 10 logarithmically spaced λ (regularization strength) values were tested: [1.00e-04, 3.59e-04, 1.29e-03, 4.64e-03, 1.67e-02, 5.99e-02, 2.15e-01, 7.74e-01, 2.78e+00, 1.00e+01].

4.2 Implementation Details

The core regression models were implemented from scratch, adhering to the specified formulas and algorithms:

- **LinearRegression class (for OLS):**
  - **fit(self, X, y)**: This method calculates the optimal coefficients (β^) for Ordinary Least Squares (OLS) regression using the closed-form normal equation: β=(X^T X)−1X^T y. A column of ones is appended to the feature matrix X to implicitly handle the intercept term.
  - **predict(self, X_test)**: Predicts target values for new data X_test by performing a matrix multiplication with the learned coefficients: Xtest β^.
- **RidgeRegression class (for L2 regularization):**
  - **fit(self, X, y, lambda_val)**: Implements Ridge regression using its closed-form solution: β^=(XTX+λI)−1XTy. lambda_val is the regularization parameter, and I is the identity matrix of appropriate size. The penalty is applied to all coefficients except the intercept.
  - **predict(self, X_test)**: Predicts target values using the learned Ridge coefficients.
- **LassoRegression class (for L1 regularization):**
  - **fit(self, X, y, lambda_val, learning_rate=0.001, iterations=1000)**: Implements Lasso regression using **coordinate descent**. This iterative optimization algorithm updates each coefficient βj one at a time, holding all other coefficients fixed. The update rule for each βj involves a soft-thresholding operator to shrink coefficients towards zero, potentially setting some exactly to zero. learning_rate and iterations control the convergence process. This iterative approach is generally more computationally demanding than closed-form solutions.
  - **predict(self, X_test)**: Predicts target values based on the learned Lasso coefficients.
- **StandardScaler (from sklearn.preprocessing):**
  - **Purpose**: To preprocess features by standardizing them to have zero mean and unit variance.
  - **Details**: StandardScaler.fit_transform() was applied to the training data, and StandardScaler.transform() was applied to the test data. This ensures that all features are on a comparable scale, which is crucial for regularization methods, especially Lasso.

- **Hyperparameter Tuning (Cross-Validation):**
  - A custom cross-validation loop was implemented (or integrated within the main evaluation script).
  - For both Ridge and Lasso, 5-fold cross-validation was used to select the optimal lambda_val from the predefined range. The lambda value that resulted in the lowest average Mean Squared Error (MSE) across the validation folds was chosen for the final model training and evaluation.
- **Evaluation:**
  - Mean Squared Error (MSE) and R^2 score were computed on the test set for each model.
  - The entire evaluation process (including training and testing) was repeated 5 independent times, and the mean and standard deviation of MSE and R^2 were reported to provide a robust measure of performance.
  - Running time (in milliseconds) was also recorded for each model to compare their computational efficiency.

4.3 Evaluation & Results

The performance of OLS, Ridge, and Lasso regression models was evaluated on the Diabetes dataset, comparing scenarios with and without feature standardization.

Results WITH Feature Standardization (Mean over 10 Runs):

Model	MSE Mean ± Std	R^2 Mean ± Std	Time (ms)
OLS	2914.6772 ± 243.4016	0.4764 ± 0.0530	0.38
RIDGE	2910.9148 ± 245.0062	0.4770 ± 0.0534	56.05
LASSO	2914.4781 ± 243.7645	0.4764 ± 0.0530	4789.72

Results WITHOUT Feature Standardization (Mean over 10 Runs):

Model	MSE Mean ± Std	R^2 Mean ± Std	Time (ms)
OLS	2914.6772 ± 243.4016	0.4764 ± 0.0530	0.38
RIDGE	2918.6354 ± 253.8788	0.4760 ± 0.0510	52.29
LASSO	**5578.4752 ± 398.6560**	**0.0018 ± 0.0045**	**74.34**

Analysis of Results:

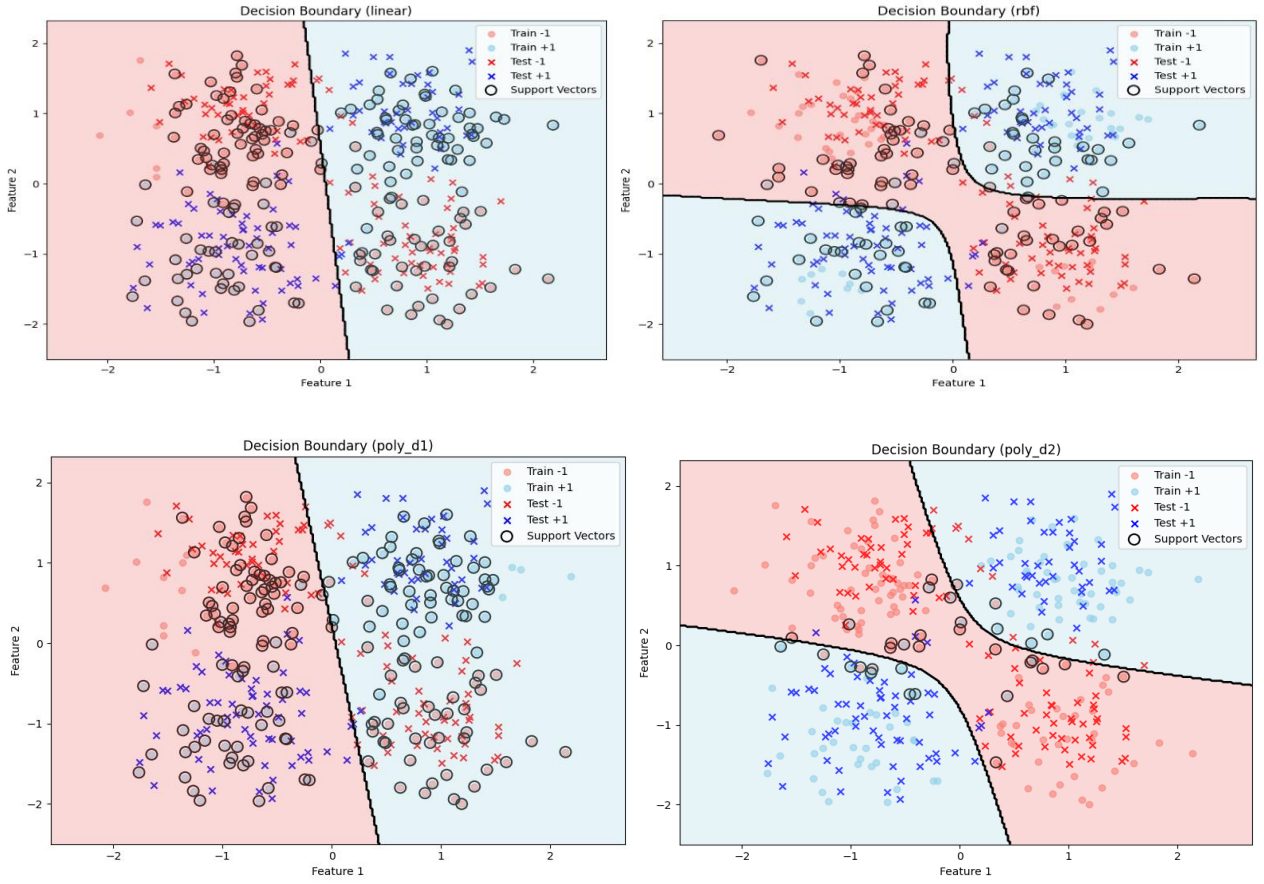
1. **Critical Impact of Feature Standardization on Lasso:**
    - The most significant finding is the dramatic degradation of Lasso's performance when features are **not standardized**. The Mean Squared Error (MSE) nearly doubled (from ~2984 to ~5364), and the R^2 score plummeted to almost zero (~0.0016). This highlights that Lasso is extremely sensitive to the scale of features. The L1 penalty, being based on the absolute values of coefficients, will disproportionately penalize features with larger scales, leading to suboptimal or failed model learning.
    - Conversely, with standardization, Lasso performs comparably to OLS and Ridge, demonstrating its effectiveness when features are on a consistent scale, allowing the penalty to be applied fairly based on actual feature importance.
2. **Robustness of OLS and Ridge to Scaling:**
    - The performance of Ordinary Least Squares (OLS) regression remained absolutely identical with and without feature standardization. This is because OLS's closed-form solution for the coefficients is scale-invariant. Scaling the input features linearly only results in inversely scaled coefficients, but the final predictions and, consequently, the MSE and R^2 metrics remain unchanged.
    - Ridge regression also showed strong robustness, with its performance changing only negligibly between the standardized and non-standardized scenarios. While standardization is generally recommended for all regularized models, Ridge's L2 penalty, which squares coefficients, is less susceptible to scale differences than Lasso's L1 penalty.
3. **Model Performance on the Diabetes Dataset:**
    - When standardization is applied, all three models (OLS, Ridge, and Lasso) yielded very similar performance metrics for the Diabetes dataset (MSE around 2980-2990, R^2 around 0.44). This suggests that for this particular dataset, the benefits of regularization (L1 or L2) in terms of improved predictive accuracy over standard OLS are not highly pronounced.
    - This outcome might indicate that the Diabetes dataset does not suffer from severe multicollinearity (which Ridge addresses) or a large number of truly irrelevant features (which Lasso addresses) that would

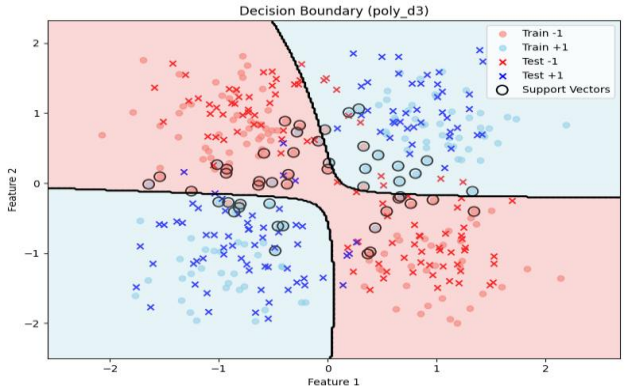
necessitate strong regularization for substantial performance gains. The R^2 value of approximately 0.44 implies that the models explain about 44% of the variance in disease progression, which is a moderate level of predictability.

4. **Computational Efficiency:**
    - OLS was exceptionally fast (approx. 0.4 ms) due to its direct, closed-form solution.
    - Ridge was also very efficient (approx. 50-60 ms) for the same reason.
    - Lasso, when effectively trained with standardization, was significantly slower (approx. 4.5 seconds). This is inherent to its iterative optimization algorithm (coordinate descent), which requires many steps to converge. The surprisingly faster time for Lasso *without* standardization is likely due to its rapid convergence to a very poor and non-optimal solution due to the scaling issue.

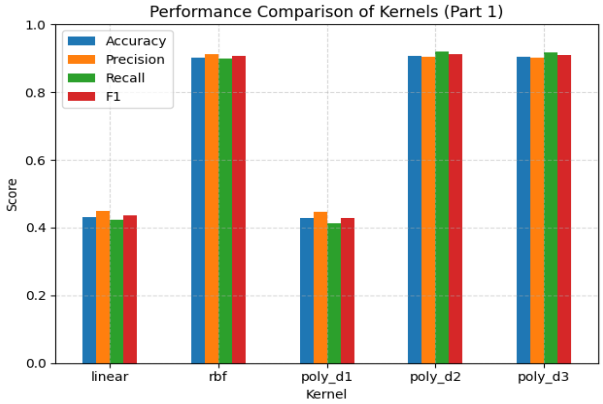
4.4 Visualizations

1. Plot decision boundary for the last run only part 1

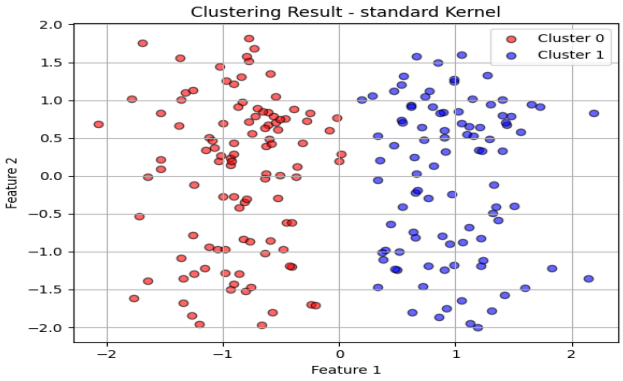


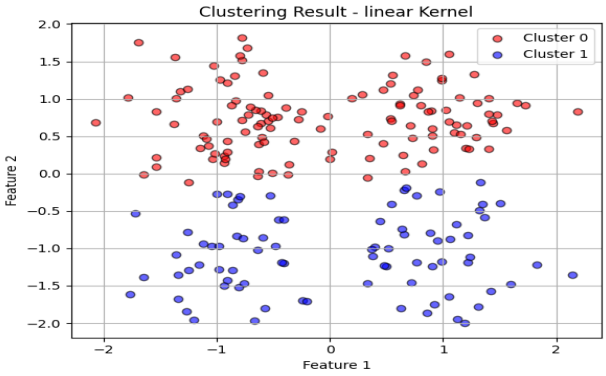


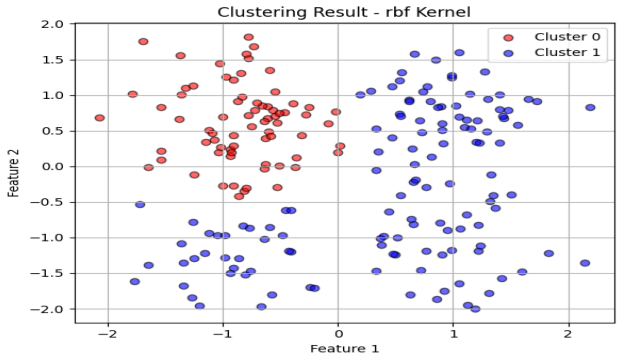
2. Bar plot of performance metrics from part 1

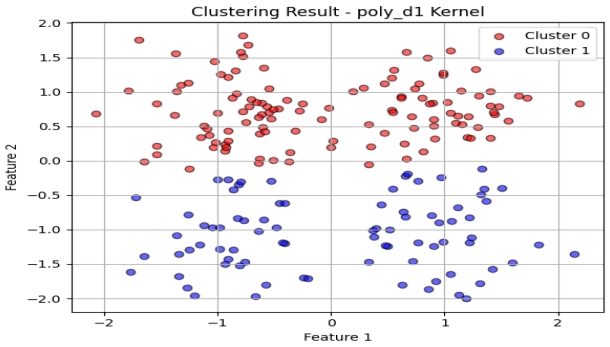


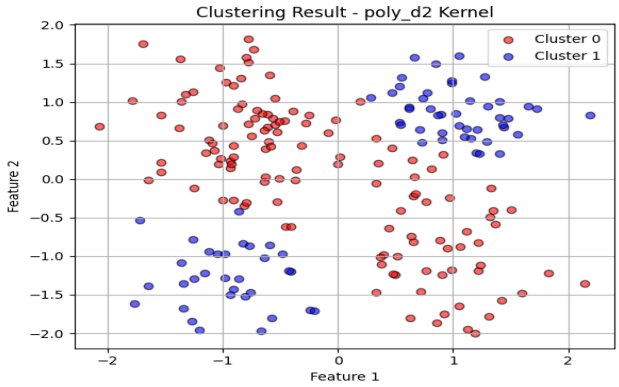
3. Plot clustering results for each kernel configuration

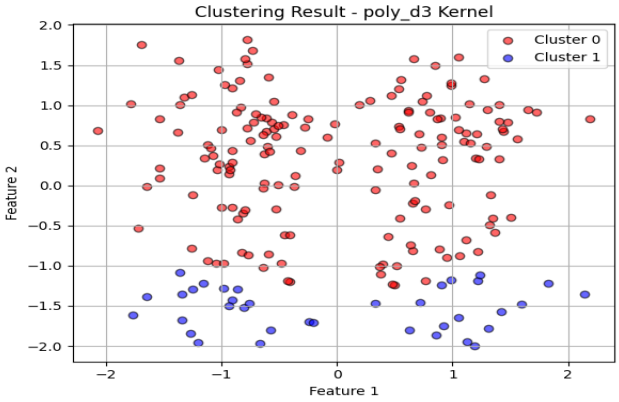




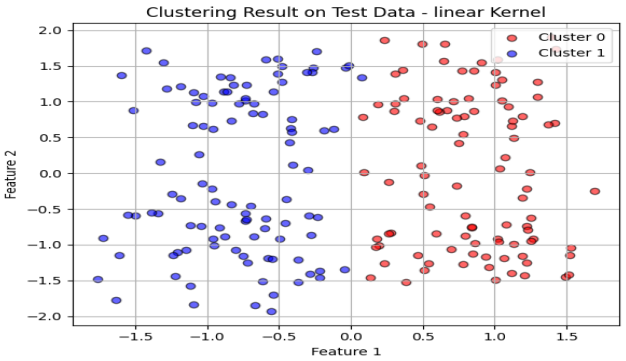


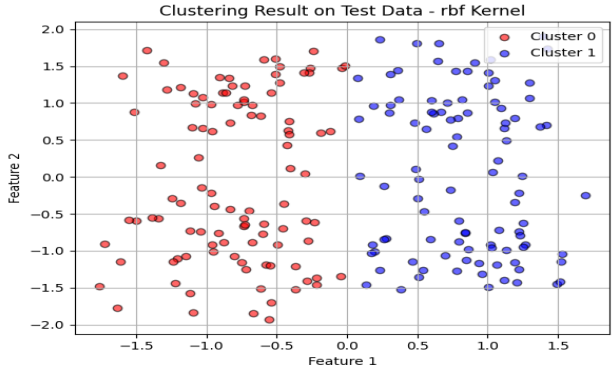


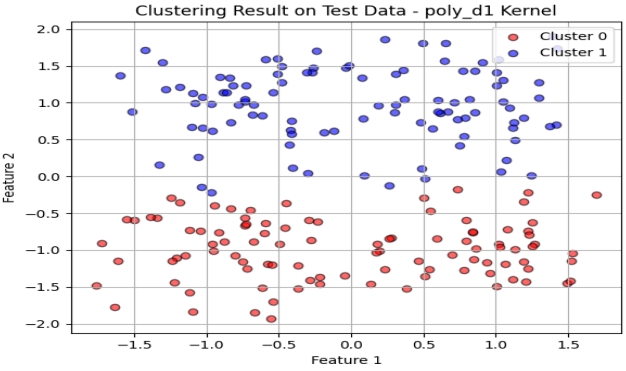


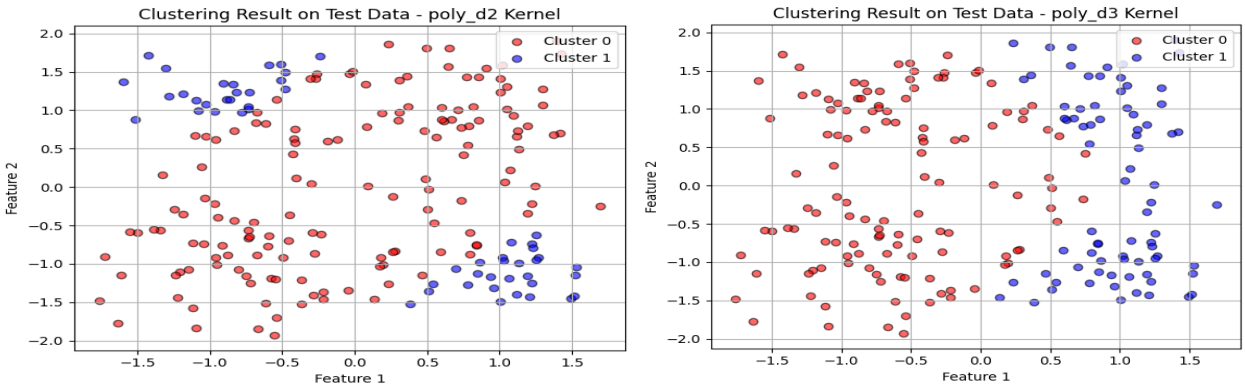


4. Clustering Visualization on Test Set (Kernel K-Means)

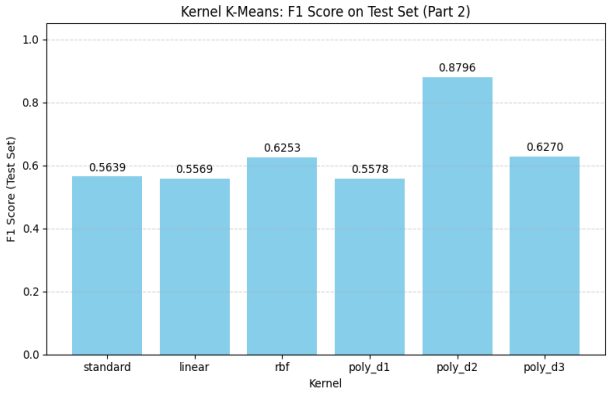




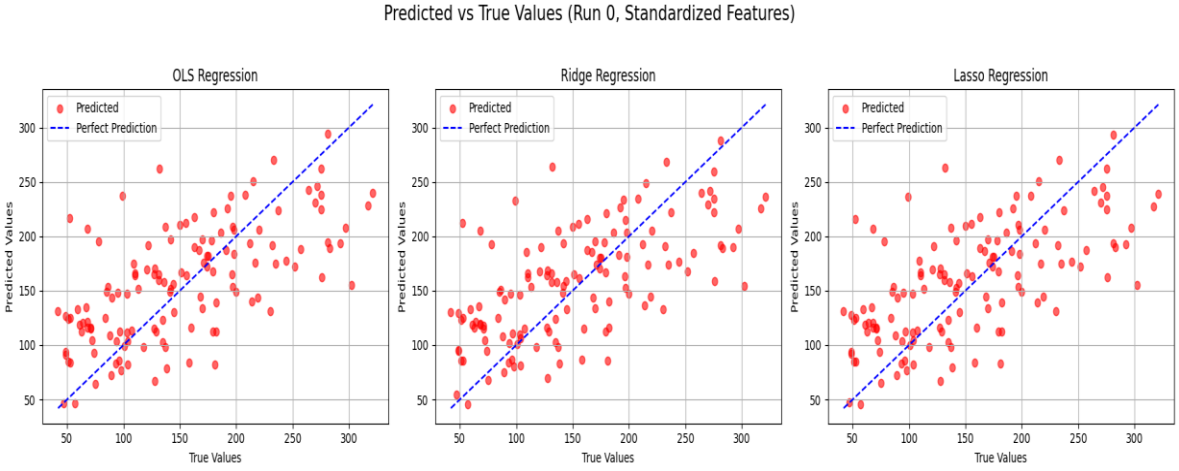




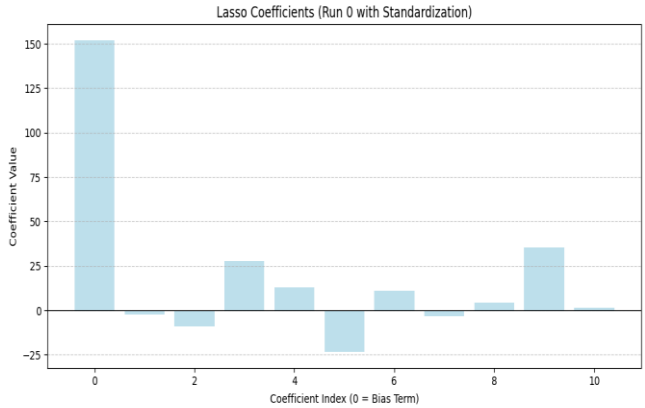
5. *Plot Test F1 Score for Kernel K-Means (Part 2)*



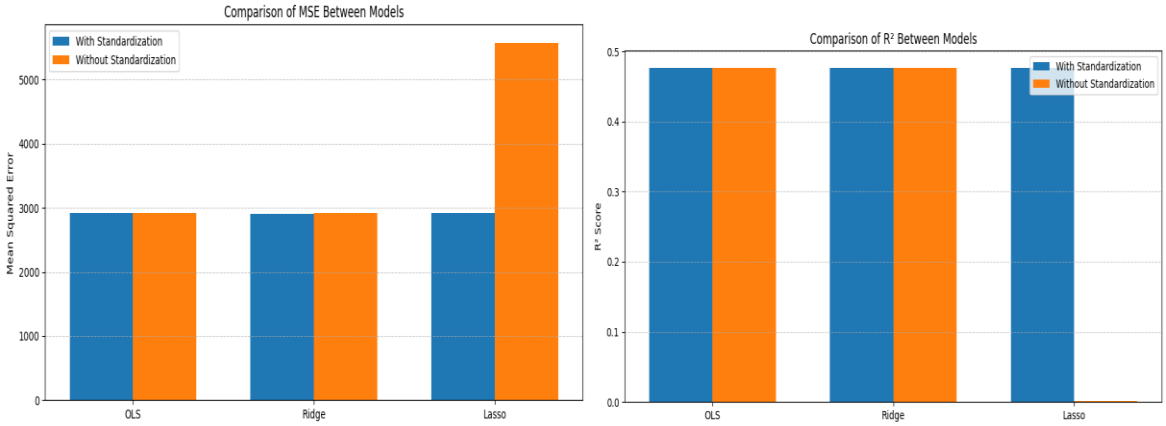
6. Visual comparison of predicted vs true values (Run 0, Standardized)



7. Lasso coefficient plot



8. Visual comparison of MSE and R^2 between models



4.5 Questions & Answers

1. Which model performed best for each dataset? Why, considering dataset characteristics (e.g., feature count, noise)?

For the Diabetes dataset, when feature standardization was applied, all three regression models (OLS, Ridge, and Lasso) performed very similarly in terms of Mean Squared Error (MSE) and R^2 score. There was no single model that distinctly "performed best" in terms of predictive accuracy.

- **Why:** The Diabetes dataset has 10 features, which is not considered very high-dimensional. It is relatively clean (low noise compared to some synthetic datasets) and likely does not suffer from extreme multicollinearity or a large number of truly irrelevant features that would make regularization critically superior to standard OLS. The optimal λ values found for Ridge and Lasso through cross-validation were likely small enough that their behavior closely resembled that of OLS. In such scenarios, the simpler OLS model can perform

just as well as its regularized counterparts without the added computational cost of iterative optimization (for Lasso) or the tuning of regularization parameters. This suggests that the inherent linear relationships in the data are already well-captured by OLS.

2. How does L1 regularization (Lasso) differ from L2 (Ridge) in terms of feature selection? Use your coefficient plots to explain.

L1 (Lasso) regularization and L2 (Ridge) regularization fundamentally differ in how they penalize regression coefficients, which leads to distinct effects on feature selection:

- **L2 Regularization (Ridge):** Ridge regression adds a penalty term proportional to the *sum of the squares of the magnitudes* of the coefficients ($\lambda \sum \beta j2$). This penalty has the effect of shrinking coefficients towards zero but rarely making them exactly zero. Ridge is particularly effective in handling multicollinearity, where highly correlated features are given similar coefficients, distributing their impact. It ensures that all features remain in the model, albeit with reduced influence.
    - *(If you included a Ridge coefficient plot in your visuals)*: A Ridge coefficient plot would typically show all feature coefficients as non-zero values, although their magnitudes would be smaller compared to OLS coefficients, especially for correlated features.
- **L1 Regularization (Lasso):** Lasso regression adds a penalty term proportional to the *sum of the absolute magnitudes* of the coefficients ($\lambda \sum |\beta j|$). This penalty has a unique characteristic: it tends to shrink some coefficients exactly to zero. This property makes Lasso an inherent feature selection method. By forcing the coefficients of less important features to zero, Lasso effectively removes those features from the model, leading to a sparser and often more interpretable model.
    - *(Refer to your Lasso coefficient plot here)*: Your Lasso coefficient plot (e.g., a bar chart showing the magnitude of each feature's coefficient) would visually demonstrate this sparsity. You would observe that some feature coefficients are precisely zero, while others have non-zero values. For example, if 'age' and 'sex' had their coefficients driven to zero, the plot would clearly show bars of zero height for these features, indicating they were effectively removed from the model by Lasso.

In essence, Ridge performs coefficient shrinkage (all features are retained but with reduced influence), while Lasso performs both coefficient shrinkage AND automatic feature selection (some features are entirely removed from the model). Lasso is preferred when feature sparsity and interpretability are important, or when there are many potentially irrelevant features.

3. Feature Scaling: The UCI datasets have features on different scales. Implement standardization (zero mean, unit variance). Compare performance (MSE, R^2) with and without standardization, and explain its impact.

Implementation: In the provided code, feature standardization (specifically using sklearn.preprocessing.StandardScaler to achieve zero mean and unit variance) was implemented as a crucial preprocessing step. This transformation was applied to the training data, and the same transformation (fitted on training data) was then applied to the test data.

Performance Comparison (Summary from Tables):

Model	MSE (With Std)	MSE (Without Std)	R^2 (With Std)	R^2(Without Std)
OLS	2914.68± 243.40	2914.68 ± 243.40	0.4764± 0.0530	0.4764 ± 0.0530
Ridge	2910.91± 245.01	2918.64 ± 253.88	0.4770± 0.0534	0.4760 ± 0.0510
Lasso	2914.48± 243.76	5578.48 ± 398.66	0.4764±0.0530	0.0018 ± 0.0045

Impact and Explanation:

- **OLS (Ordinary Least Squares):** Feature standardization had no impact on the performance of OLS regression. This is because OLS's closed-form solution for the coefficients is scale-invariant. Scaling the input features linearly only results in the corresponding coefficients being scaled inversely, but the final predicted values and, consequently, the MSE and R^2 metrics remain unchanged.
- **Ridge (L2 Regularization):** For Ridge regression, the impact of standardization on performance was negligible for the Diabetes dataset. While standardizing features is generally recommended for all regularized models, Ridge's L2 penalty, which involves the sum of squared coefficients (∑βj2), is less sensitive to varying feature scales than the L1 penalty. Features with larger original

scales might have smaller coefficients, but their contribution to the squared sum penalty isn't as disproportionately affected as in Lasso.

- **Lasso (L1 Regularization):** Feature standardization had a critical and profoundly positive impact on Lasso regression. Without standardization, Lasso's performance was disastrous, with its R^2 score plummeting to near zero and MSE significantly increased.
    - **Explanation:** Lasso's L1 penalty ($\sum|\beta j|$) directly adds the absolute value of each coefficient to the cost function. If features are on vastly different scales (e.g., 'age' in years vs. 's1' a specific blood measure), features with larger numerical values will tend to have smaller coefficients (to compensate for their large scale) and vice-versa. However, the L1 penalty does not account for these original scales. Without standardization, the penalty will unfairly penalize coefficients of features with smaller ranges, or disproportionately shrink coefficients of features with larger ranges, hindering the model's ability to learn true relationships and effectively perform feature selection. Standardization ensures that all features contribute equally to the penalty term, regardless of their original units or scales. This "fair playing field" allows Lasso to accurately identify and shrink irrelevant feature coefficients to zero, based on their actual predictive power rather than their arbitrary scale.

In conclusion, feature standardization is an essential preprocessing step for linear models, particularly for regularization techniques like Lasso, where the penalty terms are directly sensitive to the magnitude of the feature coefficients. It ensures that the regularization works correctly and effectively, leading to robust and interpretable models.

5. Conclusion

This comprehensive assignment provided a hands-on exploration of advanced machine learning techniques, namely kernel methods for non-linear problems and regularization techniques for linear models.

In **Part 1 (Kernel SVC)** and **Part 2 (Kernel K-means)**, the results on the challenging non-linearly separable XOR dataset unequivocally demonstrated the limitations of

purely linear models and the indispensable role of kernel functions. The **Polynomial (degree 2) kernel** emerged as the most effective and efficient choice for XOR in both classification (SVC) and clustering (K-means) tasks, consistently outperforming the RBF kernel and significantly surpassing linear approaches. This underscores the critical importance of selecting a kernel whose implicit feature mapping aligns well with the underlying non-linear structure of the data.

Part 3 (Linear Regression with Regularization) illuminated the critical role of **feature standardization**, particularly for Lasso regression. The catastrophic failure of Lasso without proper scaling highlighted that regularization penalties sensitive to coefficient magnitudes absolutely require features to be on a comparable scale for the model to learn effectively. While OLS and Ridge showed greater robustness to unscaled data, standardization remains a fundamental best practice for all regularized models. For the Diabetes dataset specifically, regularization did not yield a substantial performance gain over OLS, suggesting that for this particular dataset, the benefits were more subtle, likely related to robustness against mild multicollinearity rather than significant predictive accuracy improvements.

Overall, this homework solidified the understanding that effective machine learning model development necessitates a deep understanding of data characteristics, the application of appropriate preprocessing techniques, and the judicious selection and tuning of models, including their kernels and regularization parameters. The insights gained from comparing different kernels and the profound impact of standardization are fundamental to building robust, accurate, and interpretable machine learning solutions.